Sprint 12F – Why Effective Permission Calculation Is Needed

Enterprise Inventory Management API

Permission Audit, Diagnostics & Effective Authorization

---

1. Purpose of This Document

This document explains **why Sprint 12F is being introduced** into the Enterprise Inventory Management API and what architectural problem it solves.

Sprint 12A through Sprint 12E established the foundation for database-driven, permission-based authorization:

- Permission management
- Role management
- Role-permission assignment
- User-role assignment
- User-specific permission overrides

However, these capabilities primarily answer:

> **"What authorization configuration exists?"**

They do not completely answer:

> **"What is this user's final effective permission, and why?"**

Sprint 12F introduces the **calculation, visibility, diagnostics, and auditability layer** required to answer that question.

---

2. The Authorization Problem

The application has multiple sources from which a user's permissions can originate.

The current authorization model is:

```
User
 │
 ├── UserRole
 │      │
 │      ▼
```

```
  │     Role
  │      │
  │      ▼
  │ RolePermission
  │      │
  │      ▼
  │ Permission
  │
  └── UserPermission
         │
         └── Allow / Deny
```

Therefore, a user's authorization is no longer simply:

```
User → Permission
```

It becomes:

```
User
 ├── Roles
 │    └── Role Permissions
 │
 └── User-specific Permission Overrides
```

The application therefore needs a deterministic mechanism to combine these sources.

---

3. What Sprint 12A–12E Already Provide

The previous authorization sprints establish the individual pieces.

Sprint 12A – Permission-Based Authorization

Provides:

- Permission definitions
- Permission-based authorization foundation
- Authorization infrastructure

Sprint 12B – Role Management

Provides:

- Role creation

- Role update
- Role deletion
- Role retrieval

Sprint 12C – Permission Management and Role Permissions

Provides:

- Permission management
- Role-permission assignment
- Permission existence validation
- Administrator protection

Sprint 12D – User-Role Assignment

Provides:

```
User
 ↓
Role
```

A user can have multiple roles.

For example:

```
Dharmendra
 ├── Admin
 └── Operator
```

Sprint 12E – User-Specific Permission Overrides

Provides:

```
User
 ↓
UserPermission
 ↓
Allow / Deny
```

For example:

```
Dharmendra
 ├── User.Delete → DENY
 └── Report.Export → ALLOW
```

At this point the application has all the required authorization sources.

However, there is an important architectural gap.

---

4. The Gap After Sprint 12E

The application knows:

```
Dharmendra has Admin role.
Admin has User.Delete.

Dharmendra also has a UserPermission override:
User.Delete = DENY.
```

But where is the logic that determines:

```
Final decision:
User.Delete = DENY
```

And more importantly:

> **Why was it denied?**

Without Sprint 12F, the application has the data but does not have a dedicated application-level calculation model for interpreting that data.

This is the problem Sprint 12F solves.

---

5. Configuration vs Effective Authorization

This distinction is extremely important.

Authorization Configuration

The database tells us:

```
Role: Admin
Permission: User.Delete
```

and:

```
User: Dharmendra
Permission: User.Delete
Override: Deny
```

These are **authorization configuration records**.

Effective Authorization

The application must calculate:

```
Admin grants User.Delete
        +
User explicitly denies User.Delete
        ↓
Final decision
        ↓
DENY
```

This calculated result is the user's:

Effective Permission

Therefore:

```
Configured Permissions
        ↓
Permission Resolution
        ↓
Effective Permissions
```

Sprint 12F introduces this missing resolution layer.

---

6. Why We Should Not Put This Logic in the Controller

A common implementation mistake would be to write the calculation directly inside a controller:

```
Controller
 ├── Load roles
 ├── Load role permissions
 ├── Load user overrides
 ├── Resolve conflicts
 ├── Determine effective permissions
 └── Return response
```

This violates separation of concerns.

The controller should primarily handle:

```
HTTP Request
     ↓
Application Service
     ↓
HTTP Response
```

The permission calculation belongs to the application layer.

Therefore:

```
Controller
    ↓
IEffectivePermissionService
    ↓
EffectivePermissionService
    ↓
Repositories
    ↓
Database
```

This keeps authorization calculation independent from HTTP concerns.

---

7. Why a User Can Have Different Permission Sources

Consider this example.

User

```
Dharmendra
```

Roles

```
Admin
Operator
```

Admin permissions

```
User.Read
User.Create
User.Update
User.Delete
```

Operator permissions

```
Product.Read
Product.Create
Product.Update
```

User-specific overrides

```
Report.Export → ALLOW
User.Delete   → DENY
```

The final result should be:

Permission	Decision	Source
User.Read	ALLOW	Admin
User.Create	ALLOW	Admin
User.Update	ALLOW	Admin
User.Delete	DENY	User Override
Product.Read	ALLOW	Operator
Product.Create	ALLOW	Operator
Product.Update	ALLOW	Operator
Report.Export	ALLOW	User Override

Notice something important.

The final result is not simply a list of permission IDs.

We also need to know:

- Permission
- Decision
- Source
- Role
- Override type

That is why Sprint 12F requires additional resolution logic.

---

8. Permission Precedence Must Be Explicit

When multiple authorization sources exist, the application needs a deterministic rule.

For this project, the recommended precedence is:

```
User-specific DENY
        ↓
User-specific ALLOW
        ↓
Role-based permission
        ↓
No permission
```

Therefore:

Case 1

```
Role → ALLOW
User Override → nothing
```

Result:

```
ALLOW
```

Case 2

```
Role → nothing
User Override → ALLOW
```

Result:

```
ALLOW
```

Case 3

```
Role → ALLOW
User Override → ALLOW
```

Result:

```
ALLOW
```

Case 4

```
Role → ALLOW
User Override → DENY
```

Result:

```
DENY
```

This is the most important conflict scenario.

---

9. Why Sprint 12F Is More Than Another CRUD Sprint

Sprints 12A–12E are primarily about **managing authorization data**.

Sprint 12F is different.

It is about **interpreting authorization data**.

```
12A–12E
   ↓
Authorization Configuration
   ↓
12F
   ↓
Authorization Resolution
```

```
    ↓
Effective Permission
```

Therefore, Sprint 12F is an architectural layer rather than simply another collection of CRUD endpoints.

---

10. Effective Permission Calculation

Sprint 12F introduces a dedicated service:

```
public interface IEffectivePermissionService
{
    Task<IReadOnlyList<EffectivePermissionResponse>>
        GetEffectivePermissionsAsync(int userId);
}
```

The implementation will:

1. Identify the user's roles.
2. Retrieve permissions granted by those roles.
3. Retrieve user-specific permission overrides.
4. Group authorization information by permission.
5. Apply the defined precedence rules.
6. Determine the final decision.
7. Identify the source of that decision.
8. Return the effective authorization result.

---

11. Why the RolePermission Repository Needed Additional Plumbing

The existing repository method:

```
GetPermissionIdsByRoleIdAsync(int roleId)
```

returns only:

```
[1, 2, 3, 4]
```

This is sufficient for operations such as:

- assignment
- comparison
- replacement

But Sprint 12F needs more information.

For example:

```
Permission: User.Delete
Granted by: Admin
```

The calculation layer therefore needs to know:

```
RoleId
Role
PermissionId
Permission
```

This is why Sprint 12F introduces:

```
Task<List<RolePermission>> GetByRoleIdsAsync(
    IEnumerable<int> roleIds);
```

This allows the application to determine not only:

> "Does the user have this permission?"

but also:

> "Which role granted this permission?"

This information is necessary for the diagnostic and breakdown APIs.

---

12. Why Source Information Matters

Suppose the API returns:

```
{
    "permissionName": "User.Delete",
    "isAllowed": false
}
```

The result tells us the decision.

But it does not explain the decision.

An enterprise API should ideally provide:

```
{
    "permissionName": "User.Delete",
    "isAllowed": false,
    "source": "UserOverride",
    "roleName": "Admin",
    "overrideType": "Deny"
}
```

Now the administrator can understand:

```
Admin granted User.Delete
        ↓
User-specific override denied it
        ↓
Final decision = DENY
```

This is significantly more useful for troubleshooting.

---

13. Effective Permission API

Sprint 12F introduces:

```
GET /api/users/{userId}/permissions/effective
```

Example:

```
[
    {
        "permissionId": 1,
        "permissionName": "User.Read",
        "isAllowed": true,
        "source": "Role",
        "roleName": "Admin",
        "overrideType": null
    },
    {
        "permissionId": 4,
        "permissionName": "User.Delete",
        "isAllowed": false,
        "source": "UserOverride",
        "roleName": "Admin",
        "overrideType": "Deny"
```

```
    }
]
```

This API answers:

> **What permissions does this user effectively have?**

---

14. Permission Breakdown API

The effective API gives the final answer.

The breakdown API explains the authorization configuration.

```
GET /api/users/{userId}/permissions/breakdown
```

It can show:

```
User
 ├── Roles
 │    ├── Admin
 │    └── Operator
 │
 └── Permissions
      ├── User.Read → Admin → ALLOW
      ├── User.Delete → Admin → ALLOW
      └── User.Delete → User Override → DENY
```

This is particularly useful for administrators and support teams.

---

15. Permission Diagnostic API

The diagnostic API answers a very specific question:

> **Why is this permission allowed or denied?**

Example:

```
GET /api/users/5/permissions/check?permission=User.Delete
```

Response:

```
{
    "userId": 5,
    "permission": "User.Delete",
    "allowed": false,
    "reason": "Explicit user deny override",
    "sources": [
        {
            "source": "Role",
            "role": "Admin",
            "result": true
        },
        {
            "source": "UserOverride",
            "override": "Deny",
            "result": false
        }
    ]
}
```

This is extremely valuable when debugging authorization problems.

---

16. Why This Matters in Real Enterprise Systems

Imagine a production support ticket:

> "User Dharmendra cannot export reports."

Without diagnostic capabilities, a developer may need to manually investigate:

```
User
 ↓
User Roles
 ↓
Roles
 ↓
Role Permissions
 ↓
User Overrides
 ↓
Database records
 ↓
Application authorization logic
```

This can become time-consuming.

With Sprint 12F:

```
GET /api/users/5/permissions/check?permission=Report.Export
```

The system can explain:

```
Report.Export
     ↓
User Override
     ↓
ALLOW
```

or:

```
Report.Export
     ↓
No role permission
     ↓
No user override
     ↓
DENY
```

This significantly improves troubleshooting and operational visibility.

---

17. Sprint 12F Is Not a Historical Audit Log

There is an important terminology distinction.

Sprint 12F provides:

> **Authorization configuration/source auditing**

It does NOT yet provide a complete historical audit trail.

For example, Sprint 12F can answer:

```
Why is User.Delete denied right now?
```

It does not necessarily answer:

```
Who changed User.Delete from ALLOW to DENY?
When was it changed?
What was the previous value?
What is the new value?
```

A true historical audit system would require additional information such as:

```
ChangedBy
ChangedAt
OldValue
NewValue
Entity
EntityId
Action
```

That can be implemented as a future enterprise capability.

---

18. Sprint 12F Improves Maintainability

Without a centralized calculation service, authorization logic may eventually become duplicated across:

- Controllers
- Middleware
- Authorization handlers
- Services
- Background processes
- Administrative APIs

This can result in inconsistent decisions.

Sprint 12F creates a centralized concept:

```
EffectivePermissionService
```

which becomes the authoritative location for calculating effective permissions.

---

19. Sprint 12F Improves Testability

The calculation logic can be tested independently from HTTP.

For example:

```
Role Permission = ALLOW
User Override = DENY

Expected:
DENY
```

Another test:

```
Role Permission = ALLOW
User Override = none

Expected:
ALLOW
```

Another:

```
Role Permission = none
User Override = ALLOW

Expected:
ALLOW
```

This gives us a clear authorization test matrix.

---

20. Authorization Test Matrix

Role Permission	User Override	Effective Result
None	None	DENY
Allow	None	ALLOW
None	Allow	ALLOW
Allow	Allow	ALLOW
Allow	Deny	DENY
None	Deny	DENY
Multiple Roles Allow	None	ALLOW
Multiple Roles Allow	Deny	DENY

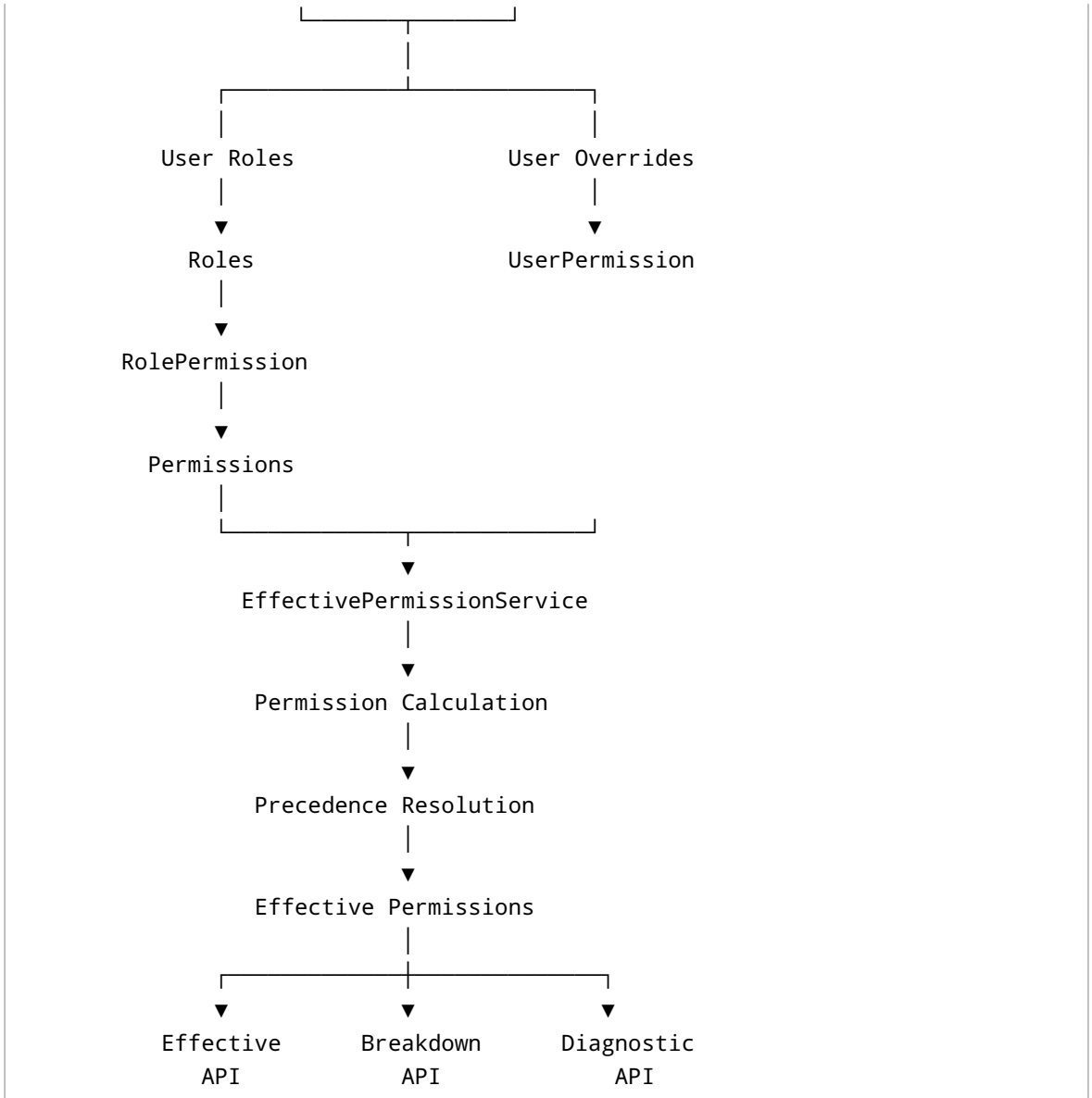
These rules become deterministic application behavior rather than undocumented assumptions.

---

21. Sprint 12F Architectural Flow

The final architecture becomes:

```
                    ┌──────────────┐
                    │     User     │
```



---

22. What Sprint 12F Does NOT Mean

Sprint 12F does not mean that Sprint 12A–12E were incomplete or incorrectly implemented.

They solved different problems.

```
12A
Permission-based authorization

12B
Role management

12C
Permission management + role permissions
```

```
12D
User-role relationships

12E
User-specific Allow/Deny overrides

12F
Effective permission resolution + visibility + diagnostics
```

Each sprint therefore has a distinct responsibility.

---

23. Is Sprint 12F Technically Mandatory?

Strictly speaking:

No.

The application can perform authorization without exposing effective-permission APIs.

For example:

```
User
 ↓
Roles
 ↓
Permissions
 ↓
Authorization Handler
 ↓
ALLOW / DENY
```

That is sufficient for a basic authorization system.

However, for this project, Sprint 12F is justified because the stated objective is to build a:

> **production-inspired enterprise application**

An enterprise authorization system benefits from being:

- Explainable
- Diagnosable
- Testable
- Auditable
- Maintainable
- Deterministic

Therefore:

> **Sprint 12F is not mandatory for authorization functionality, but it is highly valuable for enterprise authorization maturity.**

---

24. Why We Are Implementing Sprint 12F

The key architectural reason can be summarized as:

```
Sprint 12A–12E
        ↓
Store and manage authorization configuration
        ↓
       GAP
        ↓
How do we calculate the final permission?
How do we know where it came from?
How do we explain why it was denied?
How do we diagnose conflicting permissions?
        ↓
Sprint 12F
        ↓
Effective Permission Calculation
        ↓
Permission Visibility
        ↓
Permission Diagnostics
        ↓
Authorization Source Analysis
```

---

25. Final Decision

Sprint 12F will be implemented.

However, we will **not treat it as unnecessary CRUD functionality**.

The architectural purpose is:

> **To introduce a dedicated authorization resolution layer that converts multiple authorization sources into a deterministic effective permission and provides visibility into the reason and source behind that decision.**

This makes the authorization subsystem significantly more enterprise-oriented.

---

26. Sprint 12F Scope

12F.1 – Effective Permission Calculation

Implement:

```
IEffectivePermissionService
EffectivePermissionService
```

Responsibilities:

- Retrieve user roles
- Retrieve role permissions
- Retrieve user overrides
- Resolve conflicts
- Apply precedence
- Determine final permission
- Determine permission source

12F.2 – Effective Permission API

```
GET /api/users/{userId}/permissions/effective
```

12F.3 – Permission Breakdown API

```
GET /api/users/{userId}/permissions/breakdown
```

12F.4 – Permission Diagnostic API

```
GET /api/users/{userId}/permissions/check?permission=User.Delete
```

12F.5 – Permission Configuration Audit API

```
GET /api/users/{userId}/permissions/audit
```

This exposes the current authorization configuration and sources.

It does not claim to be a historical audit log.

12F.6 – Authorization Testing & Hardening

Test:

- User with no roles
- User with one role
- User with multiple roles
- Multiple roles granting the same permission
- User Allow override
- User Deny override
- Role Allow + User Deny
- Role Allow + User Allow
- No permission
- Invalid user
- Invalid permission
- Duplicate role permissions
- Conflicting authorization sources

---

27. Definition of Done

Sprint 12F will be considered complete when an administrator can answer all of the following questions:

Question 1

What permissions does this user effectively have?

Question 2

Which role or user override provided the permission?

Question 3

Is the permission ultimately allowed or denied?

Question 4

If it is denied, why?

Question 5

If multiple sources exist, which source wins?

Question 6

Can the authorization decision be tested independently?

The final capability should allow us to trace:

```
User
 ↓
Roles
 ↓
Role Permissions
 ↓
User Overrides
 ↓
Precedence Rules
 ↓
Effective Permission
 ↓
Authorization Decision
```

---

28. Final Architecture Principle

Sprint 12F introduces an important enterprise software engineering principle:

> **Authorization data and authorization decision are different concepts.**

The database stores:

```
Authorization Configuration
```

The application calculates:

```
Effective Authorization
```

And the diagnostic APIs explain:

```
Why the Effective Authorization was reached.
```

Therefore:

```
Authorization Configuration
            ↓
       Resolution Layer
            ↓
    Effective Authorization
            ↓
       Diagnostics
```

This separation is the primary architectural reason for introducing Sprint 12F.

Conclusion

Sprint 12F is not being introduced simply to add more endpoints.

It exists because the authorization model has become sufficiently complex that the system now needs a dedicated mechanism to:

1. Calculate effective permissions.
2. Resolve conflicting authorization sources.
3. Apply deterministic precedence rules.
4. Identify the source of each permission.
5. Explain authorization decisions.
6. Support production troubleshooting.
7. Improve automated authorization testing.
8. Keep authorization logic centralized and maintainable.

The goal is to evolve the project from:

```
RBAC + Permission CRUD + User Overrides
```

into:

```
Enterprise Authorization Subsystem
```

with:

```
Configuration
     +
Calculation
     +
Decision
     +
Diagnostics
     +
Auditability
```

That is the architectural justification for Sprint 12F.